

Sec 6.2. Public Key Cryptography Basics

Task 1 - Introduction

- **Authentication:** You want to be sure you communicate with the right person, not someone else pretending.
- **Authenticity:** You can verify that the information comes from the claimed source.
- **Integrity:** You must ensure that no one changes the data you exchange.
- **Confidentiality:** You want to prevent an unauthorised party from eavesdropping on your conversations.

Task 2 - Common Use of Asymmetric Encryption

Exchanging keys for symmetric encryption is a widespread use of asymmetric cryptography. Asymmetric encryption is relatively slow compared to symmetric encryption; therefore, we rely on asymmetric encryption to negotiate and agree on symmetric encryption ciphers and keys.

Answer the questions below

| In the analogy presented, what real object is analogous to the public key? Lock

Task 3 - RSA

RSA is a public-key encryption algorithm that enables secure data transmission over insecure channels. With an insecure channel, we expect adversaries to eavesdrop on it. RSA is based on the mathematically difficult problem of factoring a large number. Multiplying two large prime numbers is a straightforward operation; however, finding the factors of a huge number takes much more computing power.

Task 4 - Diffie-Hellman Key Exchange

Key exchange aims to establish a shared secret between two parties. It is a method that allows two parties to establish a shared secret over an insecure communication channel without requiring a pre-existing shared secret and without an observer being able to get this key. Consequently, this shared key can be used for symmetric encryption in subsequent communications.

Diffie-Hellman Key Exchange is often used alongside RSA public key cryptography. Diffie-Hellman is used for key agreement, while RSA is used for digital signatures, key transport, and authentication, among many others. For instance, RSA helps prove the identity of the person you're talking to via digital signing, as you can confirm based on their public key. This would prevent someone from attacking the connection with a man-in-the-middle attack. In brief, Diffie-Hellman and RSA are incorporated into many security protocols and standards to provide a comprehensive security solution.

Task 5 - SSH

Authenticating the Server: the SSH client confirms whether we recognise the server's public key fingerprint. ED25519 is the public-key algorithm used for digital signature generation and verification in this example. Our SSH client didn't recognise this key and is asking us to confirm whether we want to continue with the connection. This warning is because a man-in-the-middle attack is probable; a malicious server might have intercepted the connection and replied, pretending to be the target server.

Authenticating the Client: Now that we have confirmed that we are talking with the correct server, we need to identify ourselves and get authenticated. In many cases, SSH users are authenticated using usernames and passwords like you would log in to a physical machine. However, considering the inherent issues with passwords, this does not fall within the best security practices. At some point, one will surely hit a machine with SSH configured with key authentication instead. This authentication uses public and private keys to prove the client is a valid and authorised user on the server. By default, SSH keys are RSA keys. You can choose which algorithm to generate and add a passphrase to encrypt the SSH key.

`ssh-keygen` is the program usually used to generate key pairs. It supports various algorithms, as shown on its manual page below.

Answer the questions below

Check the SSH Private Key in ~/Public-Crypto-Basics/Task-5. What algorithm does the key use? RSA

```
user@ip-10-114-149-109:~$ ls
Crypto-Basics  John-the-Ripper  rockyou.txt
Hashing-Basics Public-Crypto-Basics
user@ip-10-114-149-109:~$ cd ~/Public-Crypto-Basics
user@ip-10-114-149-109:~/Public-Crypto-Basics$ ls
Task-5 Task-7
user@ip-10-114-149-109:~/Public-Crypto-Basics$ cd Task-5
user@ip-10-114-149-109:~/Public-Crypto-Basics/Task-5$ ls
id_rsa_1593558668558.id_rsa
user@ip-10-114-149-109:~/Public-Crypto-Basics/Task-5$
```

Task 6 - Digital Signatures and Certificates

Digital signatures provide a way to verify the authenticity and integrity of a digital message or document. Proving the authenticity of files means we know who created or modified them. Using asymmetric cryptography, you produce a signature with your private key, which can be verified using your public key. Only you should have access to your private key, which proves you signed the file. In many modern countries, digital and physical signatures have the same legal value. The simplest form of digital signature is encrypting the document with your private key. If someone wants to verify this signature, they would decrypt it with your public key and check if the files match. This process is shown in the image below.

Certificates: Prove Who You Are! : Certificates are an essential application of public key cryptography, and they are also linked to digital signatures. A common place where they're used is for HTTPS. How does your web browser know that the server you're talking to is the real tryhackme.com? The answer lies in certificates. The web server has a certificate that says it is the real tryhackme.com. The certificates have a chain of trust, starting with a root CA (Certificate Authority). From install time, your device, operating system, and web browser automatically trust various root CAs. Certificates are trusted only when the Root CAs say they trust the organisation that signed them.

Answer the questions below

What does a remote web server use to prove itself to the client? Certificate

What would you use to get a free TLS certificate for your website? Let's Encrypt

Task 7 - PGP and GPG

PGP stands for Pretty Good Privacy. It's software that implements encryption for encrypting files, performing digital signing, and more. GnuPG or GPG(opens in new tab) is an open-source implementation of the OpenPGP standard. GPG is commonly used in email to protect the confidentiality of the email messages. Furthermore, it can be used to sign an email message and confirm its integrity. Below is an example of generating GPG. You are asked about the purpose of using `gpg`, whether signing only or signing and encrypting. Besides selecting the cryptographic algorithm, we needed to choose an expiry date for the generated key. Finally, we provided some information about us: our name, email address, and a comment usually about the purpose of this key.

```
gpg --full-gen-key
gpg (GnuPG) 2.4.4; Copyright (C) 2024 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Please **select** what kind of key you want:

- (1) RSA and RSA
- (2) DSA and Elgamal
- (3) DSA (sign only)
- (4) RSA (sign only)
- (9) ECC (sign and encrypt) *default*
- (10) ECC (sign only)
- (14) Existing key from card

Your selection? 9

Please **select which** elliptic curve you want:

- (1) Curve 25519 *default*
- (4) NIST P-384
- (6) Brainpool P-256

```

Your selection? 1
Please specify how long the key should be valid.
    0 = key does not expire
    <n> = key expires in n days
    <n>w = key expires in n weeks
    <n>m = key expires in n months
    <n>y = key expires in n years
Key is valid for? (0)
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: strategos
Email address: strategos@tryhackme.thm
[...]
pub   ed25519 2024-08-29 [SC]
      AB7E6AA87B6A8E0D159CA7FFE5E63DBD5F83D5ED
uid           Strategos <strategos@tryhackme.thm>
sub   cv25519 2024-08-29 [E]

```

Practical: Now that you have your GPG key pair, you can share the public key with your contacts. Whenever your contacts want to communicate securely, they encrypt their messages to you using your public key. To decrypt the message, you will have to use your private key. Due to the importance of the GPG keys, it is vital that you keep a backup copy in a secure location.

- You would use `gpg --import backup.key` to import your key from backup.key
- To decrypt your messages, you need to issue `gpg --decrypt confidential_message.gpg`

Answer the questions below

Use GPG to decrypt the message in ~/Public-Crypto-Basics/Task-7. What secret word does the message hold? Pineapple

```

user@ip-10-114-149-109:~/Public-Crypto-Basics/Task-7$ cat tryhackme.key
cd ~/Public-Crypto-Basics/Task-7
user@ip-10-114-149-109:~/Public-Crypto-Basics/Task-7$ ls
message.gpg  tryhackme.key
user@ip-10-114-149-109:~/Public-Crypto-Basics/Task-7$ gpg --import tryhackme.key

gpg: directory '/home/user/.gnupg' created
gpg: keybox '/home/user/.gnupg/pubring.kbx' created
gpg: /home/user/.gnupg/trustdb.gpg: trustdb created
gpg: key FFA4B5252BAEB2E6: public key "TryHackMe (Example Key)" imported
gpg: key FFA4B5252BAEB2E6: secret key imported
gpg: Total number processed: 1
gpg:         imported: 1
gpg:         secret keys read: 1
gpg:         secret keys imported: 1
user@ip-10-114-149-109:~/Public-Crypto-Basics/Task-7$ gpg --decrypt message.gpg
gpg: encrypted with rsa1024 key, ID 2A0A5FDC5081B1C5, created 2020-06-30
      "TryHackMe (Example Key)"
You decrypted the file!
The secret word is Pineapple.
user@ip-10-114-149-109:~/Public-Crypto-Basics/Task-7$ S

```

Task 8 - Conclusion

- **Cryptography** is the science of securing communication and data using codes and ciphers.
- **Cryptanalysis** is the study of methods to break or bypass cryptographic security systems without knowing the key.
- **Brute-Force Attack** is an attack method that involves trying every possible key or password to decrypt a message.
- **Dictionary Attack** is an attack method where the attacker tries dictionary words or combinations of them.